

Alberi Binari

Operazioni sugli Alberi di Ricerca Binari

Questi lucidi sono basati sui lucidi di Programmazione 2 del Prof. Damiani e della Prof.ssa Baroglio

Operazioni sugli Alberi di Ricerca Binari

ARB: Algoritmi Iterativi I

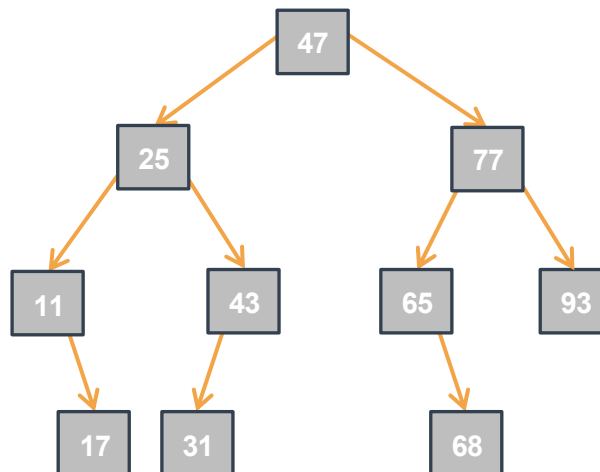


Cinque algoritmi iterativi per alberi di ricerca binari (ARB) (Binary Search Trees (BST)):

- Ricerca del minimo – `T findMinBSTiter(Tree tree);`
 - Restituisce il minimo elemento di un ARB non vuoto `tree` (termina il programma se `tree == NULL`)
- Ricerca di un elemento – `Bool searchBSTiter(IntTree tree, int value);`
 - Restituisce 1 se `value` è presente in `tree`, altrimenti 0

Minimo nel ABR?

11

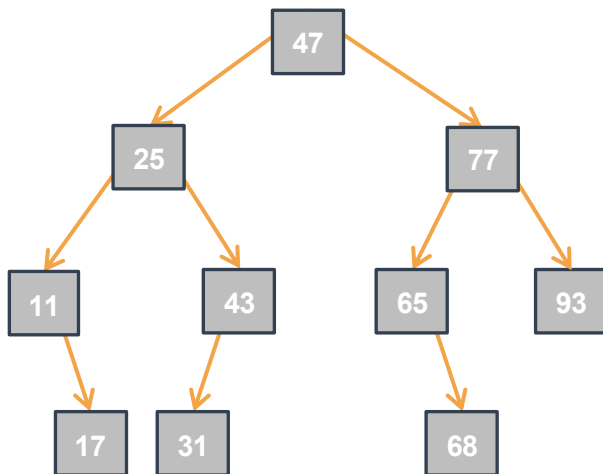


ARB: Algoritmi Iterativi II

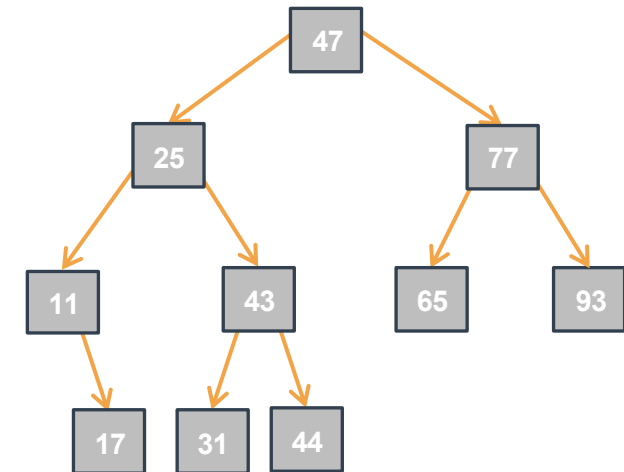


Continuazione:

- Inserimento di un elemento – Outcome `insertBSTiter(IntTree *treePtr, int value);`
 - Se `value` non presente, inserisce `value` in `*treePtr` e restituisce `INSERTED`; altrimenti non modifica `*treePtr` e restituisce `ALREADYPRESENT`; se `malloc` fallisce, restituisce `OUTOFMEMORY`.



Albero dopo aver inserito 44?

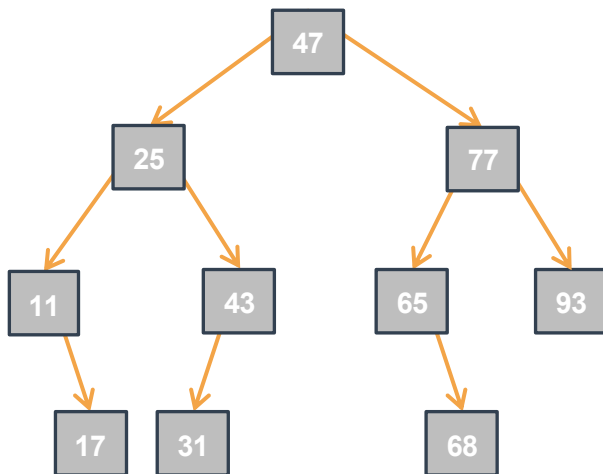


ARB: Algoritmi Iterativi III

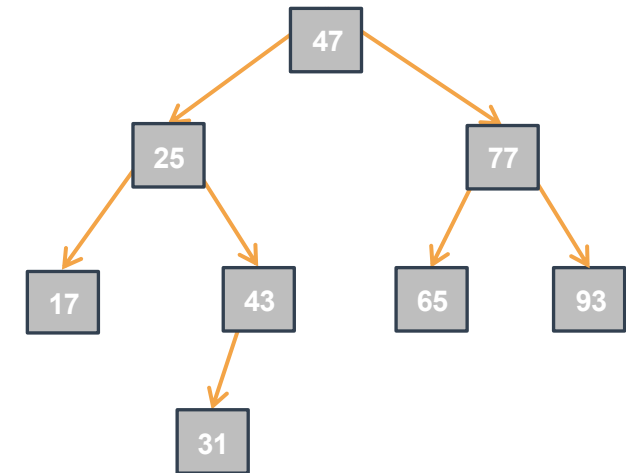


Continuazione:

- Estrazione del minimo – `T extractMinBSTiter(Tree *treePtr);`
 - Elimina e restituisce il minimo elemento di un ARB non vuoto `*treePtr` (termina il programma se `*treePtr == NULL`)



Albero dopo aver estratto 11?



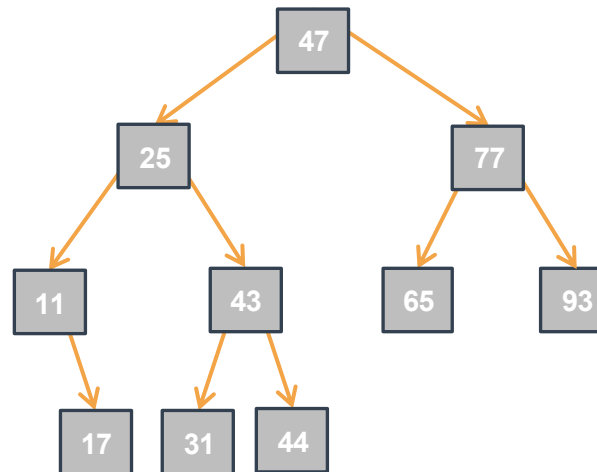
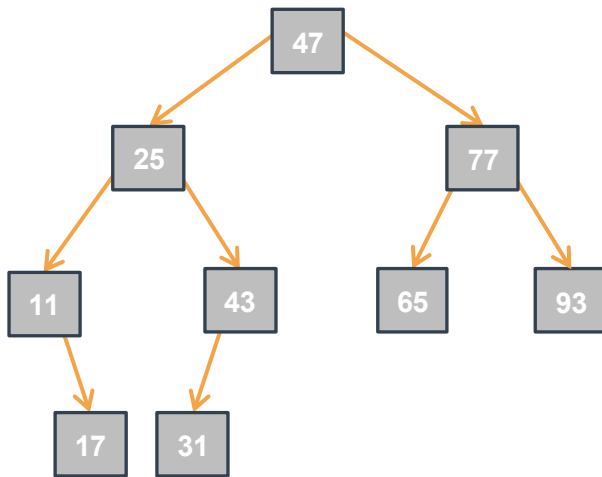
ARB: Algoritmi Iterativi IV



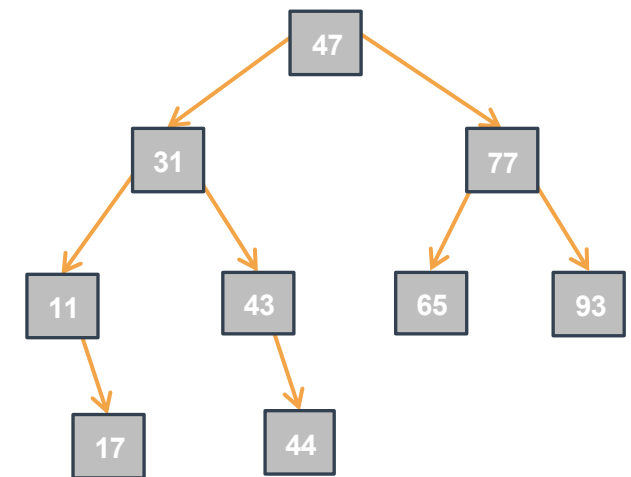
Continuazione:

- Rimozione di un elemento – `Bool removeBSTiter(IntTree *treePtr, int value);`
 - Rimuove `value` dall'ARB `*treePtr` e restituisce 1 se `value` è presente, altrimenti non modifica `*treePtr` e restituisce 0.

Albero dopo aver rimosso 44?

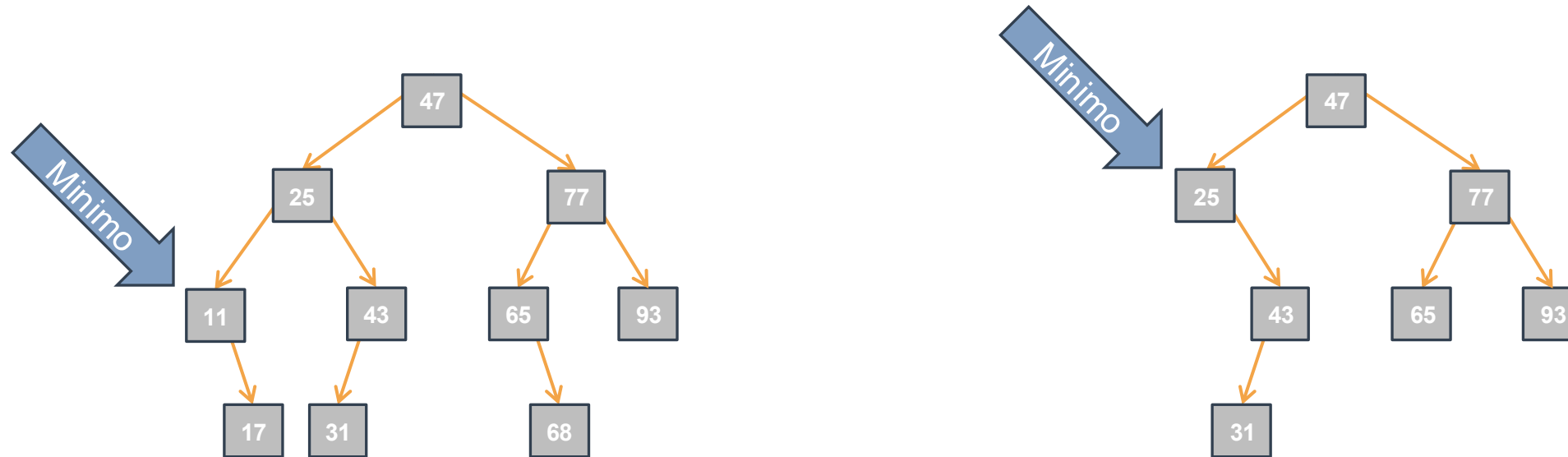


Albero dopo aver rimosso 25?



ARB: Ricerca del Minimo Iterativa I

- Dove si trova il minimo in un ARB?
 - Nel primo nodo discendente sinistro della radice che non ha il figlio sinistro.



ARB: Ricerca del Minimo Iterativa II

- `findMinBSTiter` restituisce:
 - Il minimo elemento di un ARB non vuoto `tree`
 - Termina con `exit(-1)` se `tree == NULL`
- Consideriamo (Invariante di Ciclo):
 - Il minimo di `tree` è il minimo del sottoalbero `nodePtr`
 - E se `nodePtr->left == NULL`, allora `nodePtr->data` è il minimo.

```
T findMinBSTiter(Tree tree) {  
    if (tree == NULL) { exit(-1); }  
  
    Tree nodePtr = tree;  
    while (nodePtr->left != NULL) {  
        nodePtr = nodePtr->left;  
    }  
    return nodePtr->data;  
}
```

Valore iniziale?

`tree`

Condizione?

`nodePtr->left != NULL`

Avvicinamento?

`nodePtr = nodePtr->left`

Valore dedotto?

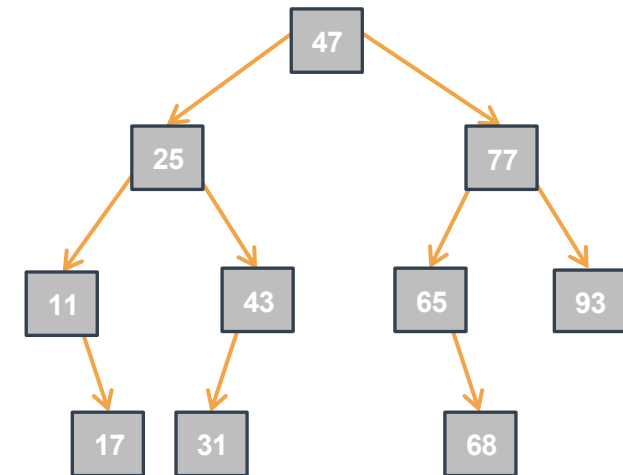
`nodePtr->data`

ARB: Ricerca del Minimo Iterativa III



- Complessità computazionale rispetto all'altezza h dell'albero?
 - Tempo:
 - Nel caso ottimo:
 - $O(1)$: il minimo è nella radice
 - Nel caso pessimo:
 - $O(h)$: il minimo è in una foglia di profondità massima
 - Spazio:
 - $O(1)$: in tutti i casi (pessimo, ottimo)

```
T findMinBSTiter(Tree tree) {  
    if (tree == NULL) { exit(-1); }  
  
    Tree nodePtr = tree;  
    while (nodePtr->left != NULL) {  
        nodePtr = nodePtr->left;  
    }  
    return nodePtr->data;  
}
```

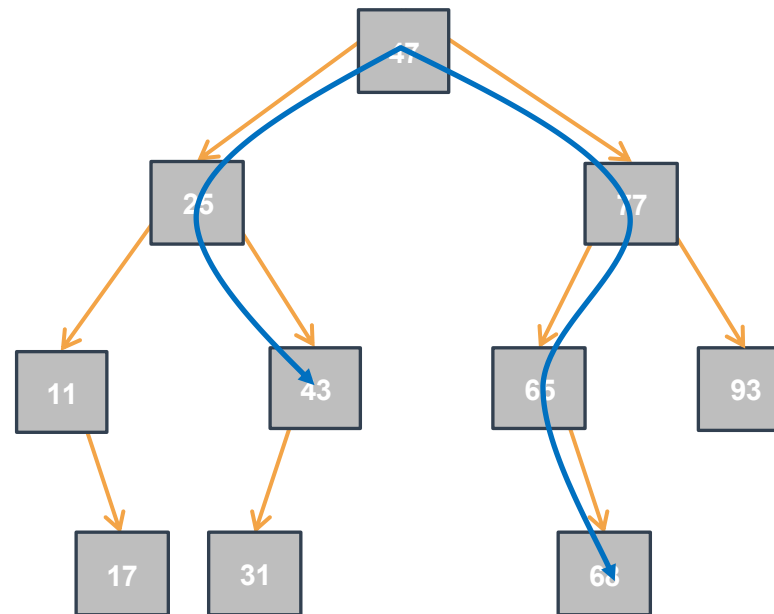


ARB: Ricerca di un Elemento Iterativa I

- Come cercare un elemento in modo efficiente in ARB?
 - Per decidere il percorso di visita basta ricordare che in un ARB, per ogni elemento (nodo) $elem$, il sottoalbero sinistro contiene elementi $\leq elem$, e il sottoalbero destro contiene elementi $\geq elem$

C'è il 43? 1

C'è il 69? 0



ARB: Ricerca di un Elemento Iterativa II



- `searchBSTiter` restituisce:
 - 1 se `value` è presente in `tree`
 - 0, altrimenti
- Consideriamo (Invariante di Ciclo):
 - `value` è presente in `tree` se-e-solo-se è presente in `nodePtr`

```
Bool searchBSTiter(IntTree tree, int value) {
    Tree nodePtr = tree;

    while (nodePtr != NULL) {
        if (value < nodePtr->data){nodePtr = nodePtr->left;}
        else if (value > nodePtr->data){nodePtr = nodePtr->right;}
        else return 1;
    }
    return 0;
}
```

Valore iniziale?

`tree`

Avvicinamento?

`sx < r < dx`

Condizione?

`nodePtr != NULL`

Valore dedotto?

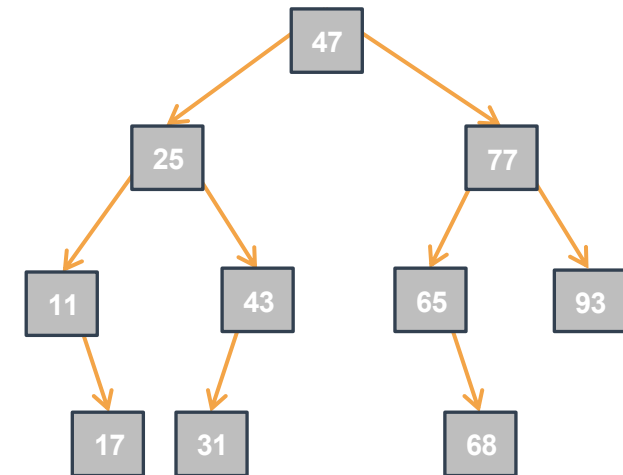
0

ARB: Ricerca di un Elemento Iterativa III



- Complessità computazionale rispetto all'altezza h dell'albero?
 - Tempo:
 - Nel caso ottimo:
 - $O(1)$: `value` è nella radice
 - Nel caso pessimo:
 - $O(h)$: `value` non è presente o presente in una foglia di profondità massima
 - Spazio:
 - $O(1)$: in tutti i casi (pessimo, ottimo)

```
Bool searchBSTiter(IntTree tree, int value) {  
    Tree nodePtr = tree;  
  
    while (nodePtr != NULL) {  
        if (value < nodePtr->data){nodePtr = nodePtr->left;}  
        else if (value > nodePtr->data){nodePtr = nodePtr->right;}  
        else return 1;  
    }  
    return 0;  
}
```



ARB: Inserimento di un Elemento Iterativa I

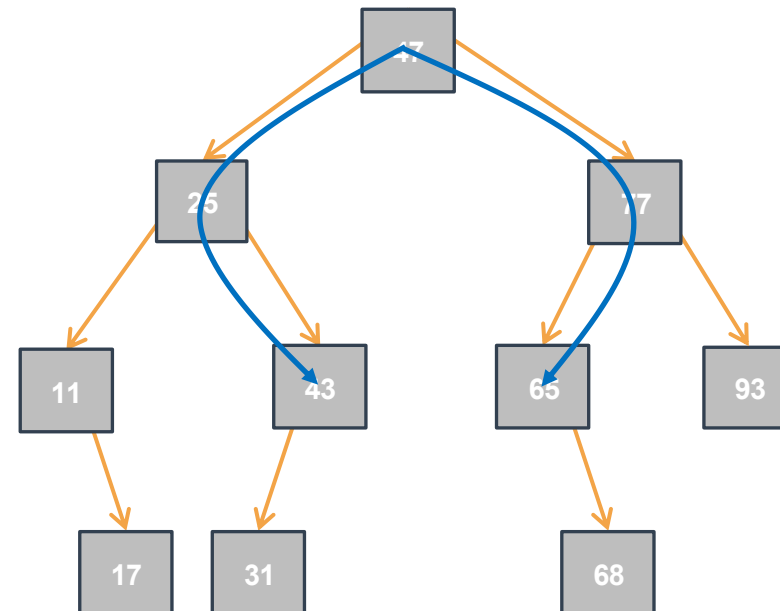
- Come cercare un elemento in modo efficiente in ARB?
 - Se già presente: segnalo solo che esiste già
 - Per decidere il percorso di visita basta ricordare che in un ARB, per ogni elemento (nodo) $elem$, il sottoalbero sinistro contiene elementi $\leq elem$, e il sottoalbero destro contiene elementi $\geq elem$

Inserisco 43?

ALREADYPRESENT

Inserisco 68?

INSERTED



ARB: Inserimento di un Elemento Iterativa II

- `insertBSTiter` restituisce:
 - `ALREADYPRESENT` se `v` è presente in `*treePtr`
 - `OUTOFMEMORY` se `malloc` fallisce altrimenti
 - `INSERTED` altrimenti
- Consideriamo (Invariante di Ciclo):
 - `v` è presente in `*treePtr` se-e-solo-se è presente in `nodePtrPtr`

```
Outcome insertBSTiter(IntTree *treePtr, int v) {
    IntTree *nodePtrPtr = treePtr;
    while (*nodePtrPtr != NULL) {
        if (v < (*nodePtrPtr)->data) {
            nodePtrPtr = &((*nodePtrPtr)->left);
        }
        else if (v > (*nodePtrPtr)->data) {
            nodePtrPtr = &((*nodePtrPtr)->right);
        }
        else { return ALREADYPRESENT; }
    }
    *nodePtrPtr = malloc(sizeof(IntTreeNode));
    if (*nodePtrPtr == NULL) { return OUTOFMEMORY; }
    (*nodePtrPtr)->data = value;
    (*nodePtrPtr)->left = NULL; (*nodePtrPtr)->right = NULL;
    return INSERTED;
}
```

Valore iniziale?

`treePtr`

Condizione?

`nodePtrPtr != NULL`

Avvicinamento?

`sx < r < dx`

Inserimento?

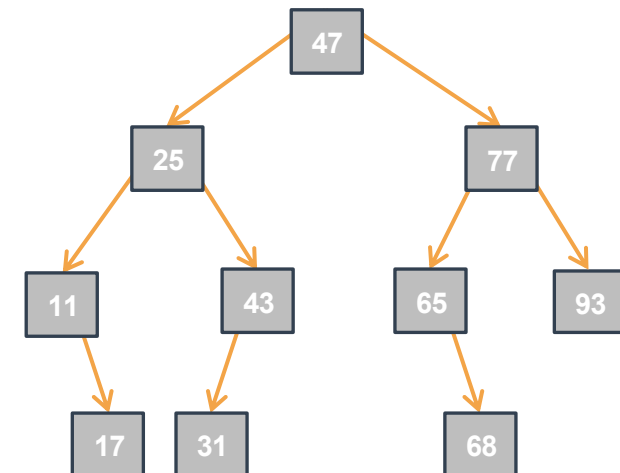
`malloc`

ARB: Inserimento di un Elemento Iterativa III



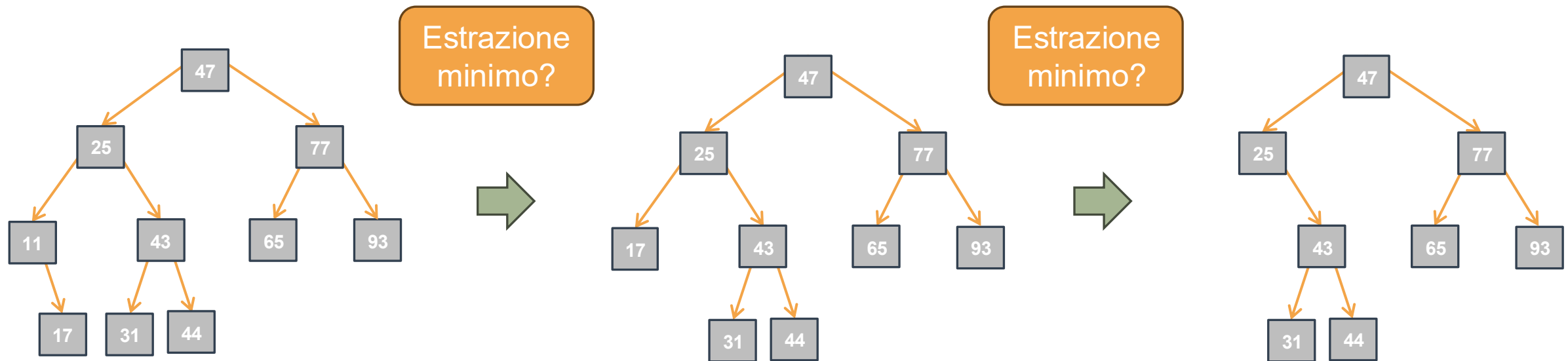
- Complessità computazionale rispetto all'altezza h dell'albero?
 - Tempo:
 - Nel caso ottimo:
 - $O(1)$: `value` è nella radice o albero vuoto
 - Nel caso pessimo:
 - $O(h)$: `value` è in una foglia di profondità massima, o non è presente e deve essere inserito come una foglia di profondità massima

```
Outcome insertBSTiter(IntTree *treePtr, int v) {
    IntTree *nodePtrPtr = treePtr;
    while (*nodePtrPtr != NULL) {
        if (v < (*nodePtrPtr)->data) {
            nodePtrPtr = &((*nodePtrPtr)->left);
        }
        else if (v > (*nodePtrPtr)->data) {
            nodePtrPtr = &((*nodePtrPtr)->right);
        }
        else { return ALREADYPRESENT; }
    }
    *nodePtrPtr = malloc(sizeof(IntTreeNode));
    if (*nodePtrPtr == NULL) { return OUTOFMEMORY; }
    (*nodePtrPtr)->data = value;
    (*nodePtrPtr)->left = NULL; (*nodePtrPtr)->right = NULL;
    return INSERTED;
}
```



ARB: Estrazione del Minimo Iterativa I

- Combinazione di ricerca il minimo e la sua rimozione
- Dove si trova il minimo in un ARB?
 - Nel primo nodo discendente sinistro della radice che non ha il figlio sinistro.



ARB: Estrazione del Minimo Iterativa II

- `extractMinBSTiter` restituisce:

- Il minimo se `*treePtr` non vuoto
- Termina il programma se `*treePtr == NULL`

- Consideriamo (Invariante di Ciclo):

- Il minimo elemento di `*treePtr` è il minimo elemento di `*nodePtrPtr`

Valore iniziale?

`treePtr`

Avvicinamento?

`nodePtrPtr =
&((*nodePtrPtr)
->left);`

Condizione?

`(*nodePtrPtr)->left
!= NULL`

Eliminazione?

`free`

```
T extractMinBSTiter(Tree *treePtr) {  
    if (treePtr == NULL || *treePtr == NULL) { exit(-1); }  
  
    Tree *nodePtrPtr = treePtr;  
    while ((*nodePtrPtr)->left != NULL) {  
        nodePtrPtr = &((*nodePtrPtr)->left);  
    }  
    Tree nodeToBeRemovedPtr = *nodePtrPtr;  
    T minimum = nodeToBeRemovedPtr->data;  
    *nodePtrPtr = nodeToBeRemovedPtr->right;  
    free(nodeToBeRemovedPtr);  
    return minimum;  
}
```

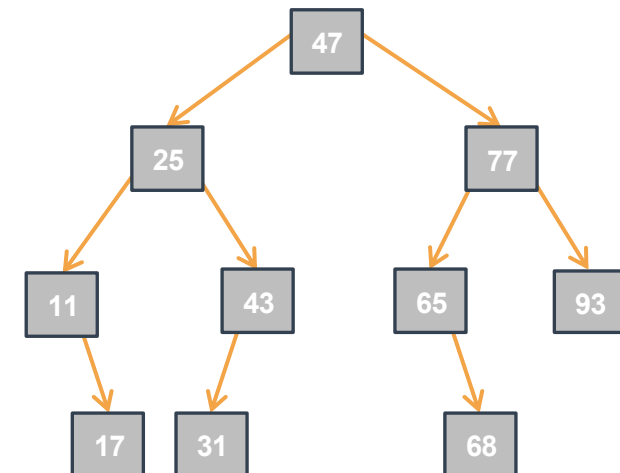


ARB: Estrazione del Minimo Iterativa III



- Complessità computazionale rispetto all'altezza h dell'albero?
 - Tempo:
 - Nel caso ottimo:
 - $O(1)$: il minimo è nella radice
 - Nel caso pessimo:
 - $O(h)$: il minimo è in una foglia di profondità massima
 - Spazio:
 - $O(1)$: in tutti i casi

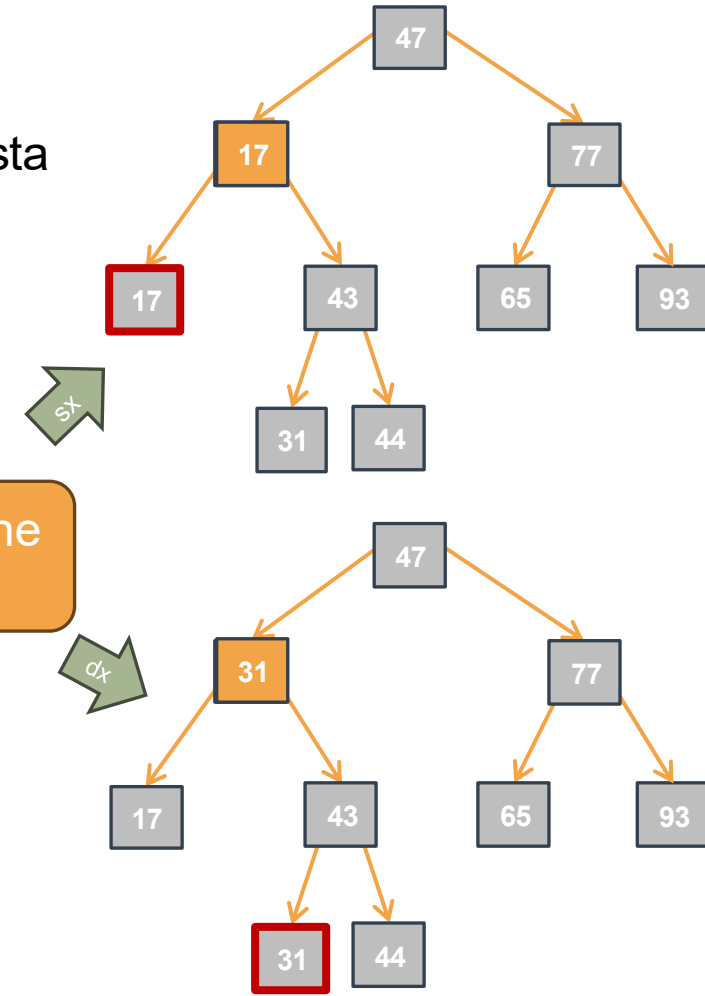
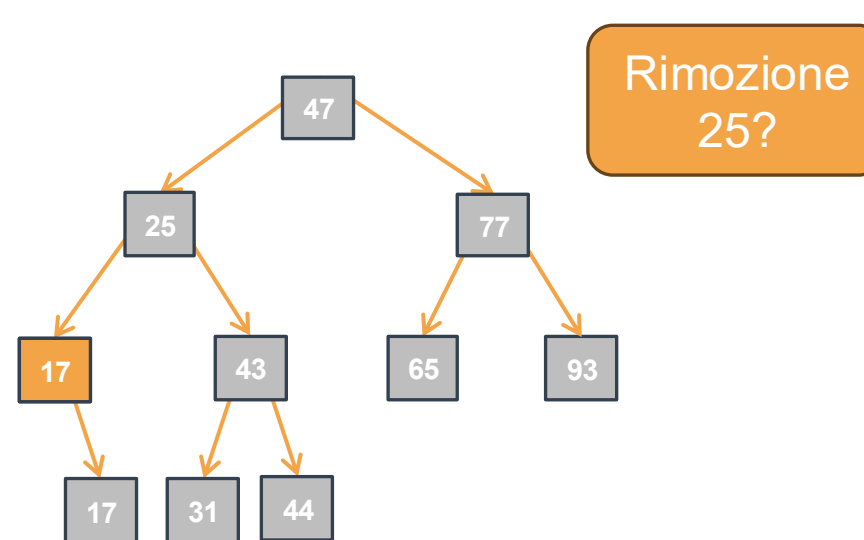
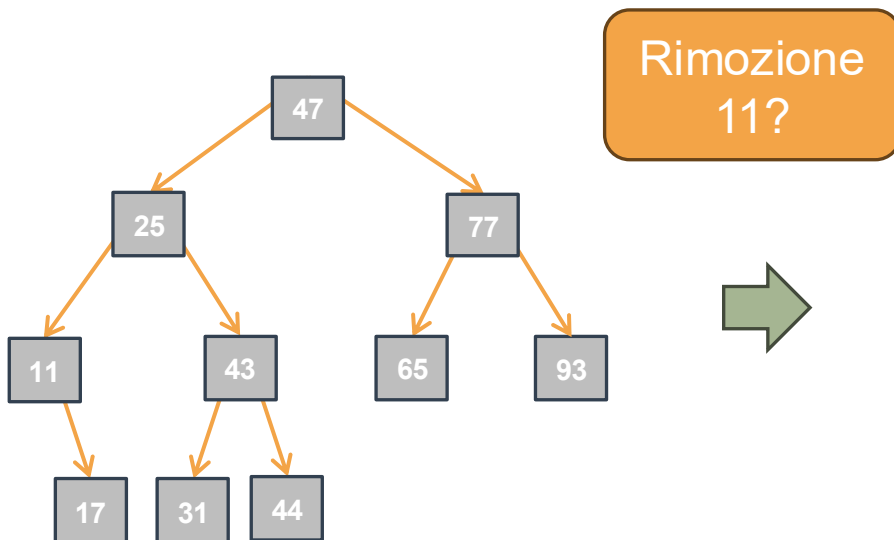
```
T extractMinBSTiter(Tree *treePtr) {  
    if (treePtr == NULL || *treePtr == NULL) { exit(-1); }  
  
    Tree *nodePtrPtr = treePtr;  
    while ((*nodePtrPtr)->left != NULL) {  
        nodePtrPtr = &((*nodePtrPtr)->left);  
    }  
    Tree nodeToBeRemovedPtr = *nodePtrPtr;  
    T minimum = nodeToBeRemovedPtr->data;  
    *nodePtrPtr = nodeToBeRemovedPtr->right;  
    free(nodeToBeRemovedPtr);  
    return minimum;  
}
```



ARB: Rimozione di un Elemento Iterativa I



- Vogliamo rimuovere l'elemento che contiene il valore `value`
- Una volta trovato, come rimuovere un nodo?
 - Caso 1: se `value` è in una foglia o in un nodo con un solo figlio, allora basta togliere il nodo dall'albero
 - Caso 2: altrimenti si può estrarre il minimo del sottoalbero destro (o il massimo del sottoalbero sinistro) e sostituirlo a `value`



ARB: Rimozione di un Elemento Iterativa II

- `removeBSTiter` restituisce:
 - 1 e modifica `*treePtr` se `v` presente
 - 0 se `v` non presente
- Consideriamo (Invariante di Ciclo):
 - `v` è presente in `*treePtr` se-e-solo-se è presente in `*nodePtrPtr`

```
_Bool removeBSTiter(IntTree *treePtr, int v) {
    IntTree *nodePtrPtr = treePtr;

    while (*nodePtrPtr != NULL) {
        if (value < (*nodePtrPtr)->data) {
            nodePtrPtr = &((*nodePtrPtr)->left); }
        else if (value > (*nodePtrPtr)->data) {
            nodePtrPtr = &((*nodePtrPtr)->right); }
        /* Caso 1 */
        /* Caso 2 */
    }

    return /* Valore dedotto da (I.C. && !Condizione) */;
}
```

Valore iniziale?

`treePtr`

Condizione?

`(*nodePtrPtr) != NULL`

Avvicinamento?

seguo albero finché `v != data`

ARB: Rimozione di un Elemento Iterativa III



- Caso Non-foglia:
 - Si può estrarre il minimo del sottoalbero destro e sostituirlo a value
- Caso Foglia (o figlio singolo):
 - basta togliere il nodo dall'albero

```
_Bool removeBSTiter(IntTree *treePtr, int v) {  
    [...]  
    /* Caso Non-foglia */  
    else if (((*nodePtrPtr)->left != NULL) &&  
            ((*nodePtrPtr)->right != NULL)) {  
        (*nodePtrPtr)->data =  
            extractMinBSTiter(&((*nodePtrPtr)->right));  
        return 1;  
    }  
    [...]  
}  
  
return /* Valore dedotto da (I.C. && !Condizione) */;  
}
```

Caso non-foglia?

```
(*nodePtrPtr)->left != NULL) && ((*nodePtrPtr)->right != NULL))
```

ARB: Rimozione di un Elemento Iterativa IV



- **Caso Non-foglia:**
 - Si può estrarre il minimo del sottoalbero destro e sostituirlo a value
- **Caso Foglia (o figlio singolo):**
 - basta togliere il nodo dall'albero

```
_Bool removeBSTiter(IntTree *treePtr, int v) {  
    [...]  
    /* Caso Foglia e Figlio singolo */  
    else {  
        Tree nodeToBeRemovedPtr = *nodePtrPtr;  
        if ((*nodePtrPtr)->left == NULL) {  
            *nodePtrPtr = (*nodePtrPtr)->right; }  
        else { *nodePtrPtr = (*nodePtrPtr)->left; }  
        free(nodeToBeRemovedPtr);  
        return 1;  
    }  
}  
return 0;  
}
```

Caso figlio singolo?

Sposto figlio

Elimina nodo?

free

Valore dedotto?

0

ARB: Rimozione di un Elemento Iterativa V



- Complessità computazionale rispetto all'altezza h dell'albero?
 - Tempo:
 - Nel caso ottimo:
 - $O(1)$: albero vuoto o v nella radice
 - Nel caso pessimo:
 - $O(h)$: v è in una foglia di profondità massima, o ha due figli e il minimo del suo sottoalbero destro è in una foglia di profondità massima, o non è presente e se lo fosse sarebbe in una foglia di profondità massima
 - Spazio:
 - $O(1)$: in tutti i casi

```
_Bool removeBSTiter(IntTree *treePtr, int v) {
    IntTree *nodePtrPtr = treePtr;
    while (*nodePtrPtr != NULL) {
        if (value < (*nodePtrPtr)->data) {
            nodePtrPtr = &((*nodePtrPtr)->left); }
        else if (value > (*nodePtrPtr)->data) {
            nodePtrPtr = &((*nodePtrPtr)->right); }
        else if (((*nodePtrPtr)->left != NULL) &&
            ((*nodePtrPtr)->right != NULL)) {
            (*nodePtrPtr)->data =
extractMinBSTiter(&((*nodePtrPtr)->right));
            return 1;
        } else {
            Tree nodeToBeRemovedPtr = *nodePtrPtr;
            if ((*nodePtrPtr)->left == NULL) {
                *nodePtrPtr = (*nodePtrPtr)->right; }
            else { *nodePtrPtr = (*nodePtrPtr)->left; }
            free(nodeToBeRemovedPtr);
            return 1;
        }
    }
    return 0;
}
```

ABR: Algoritmi Ricorsivi I



Cinque algoritmi ricorsivi per alberi di ricerca binari (ARB) (Binary Search Trees (BST)):

- Ricerca di un elemento – `Bool searchBST(IntTree tree, int value);`
 - Restituisce 1 se `value` è presente in `tree`, altrimenti 0
- Inserimento di un elemento – `Outcome insertBST(IntTree *treePtr, int value);`
 - Se `value` non è presente, inserisce `value` in `*treePtr` e restituisce `INSERTED`; altrimenti non modifica `*treePtr` e restituisce `ALREADYPRESENT`; se `malloc` fallisce, restituisce `OUTOFMEMORY`.

ARB: Ricerca di un Elemento Ricorsiva I



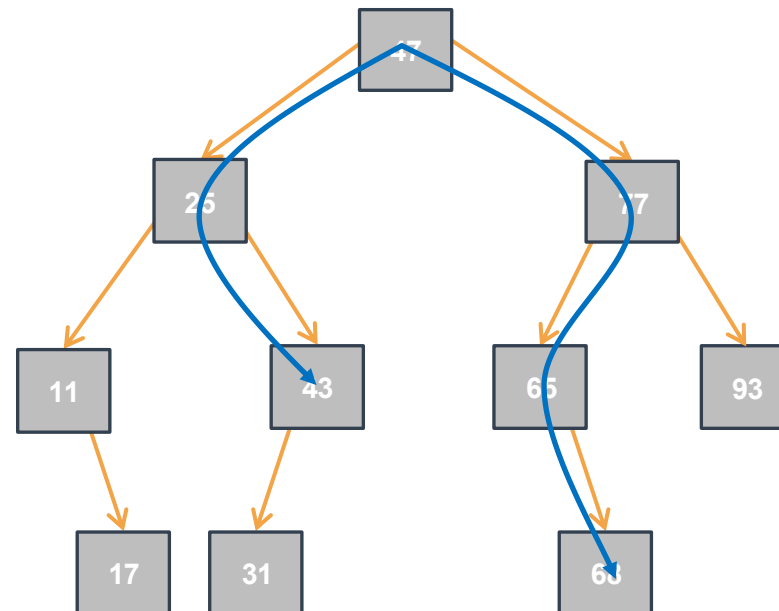
- Ripasso: Come cercare un elemento in modo efficiente in ARB?
 - Per decidere il percorso di visita basta ricordare che in un ARB, per ogni elemento (nodo) $elem$, il sottoalbero sinistro contiene elementi $\leq elem$, e il sottoalbero destro contiene elementi $\geq elem$

C'è il 43?

1

C'è il 69?

0



ARB: Ricerca di un Elemento Ricorsiva II



- Restituisce:
 - 1: se value presente in tree
 - 0: altrimenti
 - Assume che tree è un ARB
- Complessità computazionale:
 - Tempo:
 - $O(1)$ nel caso ottimo: valore nella radice
 - $O(h)$ nel caso pessimo: valore non è presente o è nella foglia di profondità massima
 - Spazio:
 - $O(h)$ oppure $O(1)$ se c'è la sibling call optimization

```
_Bool searchBST(IntTree tree, int v) {  
    if (tree == NULL) {  
        return 0;  
    } else if (v < tree->data) {  
        return searchBST(tree->left, v);  
    } else if (v > tree->data) {  
        return searchBST(tree->right, v);  
    } else {  
        return 1;  
    }  
}
```

ARB: Inserimento di un Elemento Ricorsiva I



- Restituisce:
 - INSERTED: **se** v è stato inserito in in tree
 - ALREADYPRESENT: **se** v è già presente
 - OUTOFMEMORY: **se** malloc fallisce
 - Assume che tree è un ARB
- Complessità computazionale:
 - Stessa di `searchBST`

```
Outcome insertBST(IntTree *treePtr, int v) {
    if (*treePtr == NULL) { /* Caso base 1 */
        *treePtr = malloc(sizeof(IntTreeNode));
        if (*treePtr == NULL) { return OUTOFMEMORY; }
        (*nodePtrPtr)->data = value;
        (*treePtr)->left = NULL;
        (*treePtr)->right = NULL;
        return INSERTED;
    } else if (v < (*treePtr)->data) { /* Caso ric */
        return insertBST(&(*treePtr)->left, v);
    } else if (v > (*treePtr)->data) { /* Caso ric */
        return insertBST(&(*treePtr)->right, v);
    } else { /* Caso base 2 */
        return ALREADYPRESENT;
    }
}
```

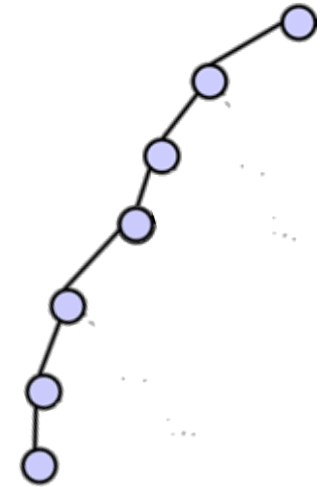
Algoritmi: Iterativi vs Ricorsivi



- In generale, la versione ricorsiva degli algoritmi che operano su strutture dati ricorsive è più facile da leggere e scrivere
 - Assieme alla dimostrazione di correttezza, usando il principio di induzione strutturale
- La versione iterativa può avere complessità computazionale in spazio minore, ed essere più performante
 - Cercate di usare la ricorsione di coda e controllate quali ottimizzazioni supporta il vostro compilatore C.

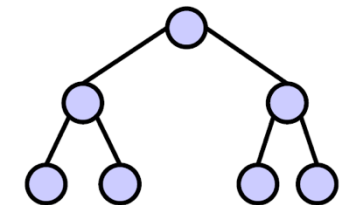
ARB: Bilanciamento

- Molte operazioni hanno complessità $O(h)$ dove h è l'altezza dell'albero
 - h è data dalla profondità massima dei nodi: vogliamo avere alberi in cui le foglie sono il più possibile tutte alla stessa profondità
- L'altezza di un ARB creato per inserimenti successivi con la funzione `insertBSTiter/insertBST` (o rimozioni arbitrarie) può variare molto a seconda dell'ordine degli elementi
 - Ad esempio: inserendo gli elementi in modo strettamente crescente si ottiene un albero completamente sbilanciato a destra.
 - Albero completamente sbilanciato ha altezza $h = n-1$
 - Albero quasi binario completo ha altezza $h = \lfloor \log_2 n \rfloor$



Sbilanciato a sinistra:

- $n = 7$
- $h = 6$



Completo:

- $n = 7$
- $h = 3$



ARB Bilanciati



- Esistono delle versioni bilanciate degli ARB
 - Ovvero ARB in cui le operazioni di inserimento, cancellazione e ricerca hanno sempre complessità $O(\log_2 n)$
 - In particolare, le operazioni di inserimento e cancellazione cambiano la struttura dell'ARB.
- Le versioni più famose di tali ARB sono (in ordine di apparizione):
 - Gli alberi AVL (AVL trees) inventati nel 1962 da Georgy Adelson-Velsky e Evgenii Landis
 - Gli alberi rosso-neri (red-black trees) inventati nel 1978 da Leonidas J. Guibas e Robert Sedgwick
 - Qui accenniamo soltanto alla loro esistenza e li vedrete probabilmente nell'insegnamento di Algoritmi e Strutture Dati.